

WILEY

INTERNATIONAL
TRANSACTIONS
IN OPERATIONAL
RESEARCHIntl. Trans. in Op. Res. 33 (2026) 297–326
DOI: 10.1111/itor.13493

A three-dimensional multiple containers loading approach considering weak heterogeneous containers for multiple customers

Hongbing Yang^a, Yue Hu^a, Wei Zhou^a, Yue Gao^{a,*}  and Junwei Wang^b^a*School of Mechanical and Electrical Engineering, Soochow University, Sozhou, China*^b*College of Information Science and Engineering, Northeastern University, Shenyang, China*E-mail: yanghongbing@suda.edu.cn [Yang]; huyue_suda@sina.com [Hu]; 382579236@qq.com [Zhou];
yuegao@suda.edu.cn [Gao]; wangjw@ise.neu.edu.cn [Wang]

Received 25 August 2023; received in revised form 23 May 2024; accepted 23 May 2024

Abstract

The container loading plan significantly affects the transportation cost of manufacturing companies. However, the traditional approaches for three-dimensional single container loading problems cannot be directly applied to three-dimensional multiple containers loading problems (3D-MCLP) due to the different basic features between the two problems; in addition, the existing solution algorithms for a 3D-MCLP have limited efficiency and quality when dealing with nonidentical boxes corresponding to multiple customers. Considering a general 3D-MCLP with weak heterogeneous containers, this paper establishes a general mathematical model and proposes an integrated greedy algorithm, multilayer tree search, and large neighborhood search algorithm to maximize the average filling rate of containers. An integrated greedy heuristic multilayer tree search algorithm is proposed to obtain the initial solution of a 3D-MCLP. Then, a large neighborhood search algorithm containing five removal operators and two repair operators is further designed to improve the results. Numerical experiments with 2014/2015 ESICUP challenge datasets and real-world case studies validate the effectiveness and efficiency of the proposed approach.

Keywords: three-dimensional multiple containers loading; weak heterogeneous containers; Last in first out constraint greedy heuristic and multilayer tree search; large neighborhood search

1. Introduction

Container loading directly affects transportation cost and distribution efficiency. Traditional container loading models are designed based on one dimension or two dimensions, where the overall volume or two-dimensional (2D) size constraint is assessed, respectively. The three-dimensional container loading problem (3D-CLP) is a natural extension of the 2D container loading problem,

*Corresponding author.

which means loading a set of rectangular boxes with a given length, width, and height into one or more containers. Compared with one- and two-dimensional models, 3D-CLP models have the advantage of broader applications. However, the third dimension causes much complexity in vertical support, load stability, shipping priority, and weight distribution. In addition, the container loading problem has progressed from single container loading to multicontainer loading, which correspondingly requires additional consideration of container selection and order splitting (i.e., boxes belonging to the same customer may be placed in multiple containers), which brings challenges in determining the number of containers and planning the container loading using a typical 3D-CLP model. Thus, an approach for a three-dimensional multiple containers loading problem (3D-MCLP) considering nonidentical items is desired to obtain the optimal results.

Generally, one essential step to solving a 3D-MCLP is establishing an optimization model considering its critical characteristics. Researchers have shown an increased interest in 3D-CLP modeling. Bortfeldt and Wäscher (2013) reviewed the constraints of variable 3D-CLPs. Then, Zhao et al. (2014) added on this basis and focused on the design and implementation of solutions. Chen et al. (1995) provided the first mixed integer programming (MIP) model for a 3D-CLP considering the relative position of items. Still, the method lacks an efficient solution procedure for the MIP model without orientation restriction. Then, Wu et al. (2010) extended the work to allow the orientation of items. Some scholars have tried to improve models considering the model complexity and effectiveness trade-offs. Hifi et al. (2010) provided several lower bounds based on linear programming relaxation and efficiency inequalities to reduce the computational time. Paquay et al. (2016) considered load-bearing, object orientation, container shape, weight distribution, and stability. In terms of unloading, Bischoff et al. (1999) proposed multidrop constraints to ensure that one customer's cargo can be unloaded without moving another customer's cargo. Regarding the multidrop constraint, some studies set it as a hard constraint (Christensen and Rousøe, 2009; Giménez-Palacios et al., 2023), but Bonet et al. (2023) believed that setting the multidrop constraint as a hard constraint would limit the flexibility of vehicle capacity, so they set the multidrop constraint as a soft constraint. In addition, the items should follow the principle of first in and last out (LIFO) based on their transportation distances.

Another main step to solving a 3D-MCLP is designing a solution algorithm, which is also the main challenge in finding the optimal loading plan. Traditional accurate algorithms show efficiency and effectiveness when addressing a loading problem with a small number of items and a single container. However, a 3D-MCLP is a typical nondeterminism polynomial complete problem, and combinatorial explosions usually occur when solved by accurate algorithms. Currently, large-sized 3D-CLPs can only be solved heuristically. According to the boxes' construction type, the heuristic algorithms can be divided into six types: wall building, stack building, horizontal layer building, block building, complete cutting, and global building (Huang and He, 2009; Lim et al., 2012; Lin et al., 2013; Rajaei et al., 2024). These methods are commonly used in determining initial solutions. Heuristic algorithms can be divided into traditional heuristic algorithms, tree search algorithms, and meta-heuristic algorithms (Fanslau and Bortfeldt, 2010; Jamrus and Chien, 2016; Zhang et al., 2022). Using tree search and meta-heuristics usually provides better results with less computational cost, broadening the search space and yielding more significant benefits. For the tree search method, Eley (2002) proposed the isomorphic block, designed a greedy heuristic algorithm to generate the block arrangement and then improved the solution provided by the greedy heuristic algorithm through tree search. Based on the concept of a block, Zhang et al. (2012) presented an improved

compound block and designed a heuristic block loading algorithm for the multilayer search. Liu et al. (2017) advanced a heuristic binary tree search algorithm to ensure the complete support and placement direction constraints of the 3D packing problem. Recently, a rising number of studies have focused on solving container loading problems using meta-heuristic algorithms, such as genetic algorithms (Gehring and Bortfeldt, 2002), tabu search algorithms (Liu et al., 2011), simulated annealing algorithms (Mack et al., 2004; Liu et al., 2017), and greedy randomized adaptive search algorithms (Moura and Oliveira, 2005; Parreño et al., 2008). Zhang et al. (2022) also proposed a hybrid heuristic algorithm by combining the ant colony algorithm and constructed a greedy algorithm to speed up the optimization process. Specially, for the multicontainer packing problem, the research results of Giménez-Palacios et al. (2023) showed that order splitting can have a strong impact on determining the number of containers. Tian et al. (2016) proposed an algorithm based on historical data to adjust the number of products in order to maximize the container loading rate, which can reduce the unit transportation cost. Zhu et al. (2012) used the column generation algorithm to improve the solving efficiency of the loading problem. In addition, some scholars have integrated vehicle routing and container loading when addressing a 3D-CLP. They have solved the vehicle routing and container loading problem sequentially and have checked the feasible packing algorithm according to the vehicle route (Wei et al., 2014).

Researchers have made a lot of efforts to solve container loading problems (Bischoff and Ratcliff, 1995; Gonçalves and Resende, 2013; Li and Zhang, 2015; Alonso et al., 2017, 2020). However, the existing studies on a 3D-MCLP have focused on identical containers and taken huge computational costs to output the optimal solution, which cannot meet the requirements of a real-world loading scenario. This paper proposes a general 3D-MCLP model and the corresponding solution algorithm to address these limitations. The proposed method is then validated using a real-world loading case.

The rest of the paper is organized as follows. Section 2 shows the 3D-MCLP model. Section 3 introduces the initial solution algorithm based on a greedy heuristic multilayer tree search, followed by the improved algorithm designed based on a large neighborhood heuristic search to improve the solution in Section 4. Section 5 shows the numerical results of case studies. Section 6 concludes the paper and presents some contributions.

2. A general three-dimensional multiple containers loading model

2.1. Parameters and variables

The notation of parameters and variables used in this paper is listed below:

R	The average filling rate of containers
i	The index of box types, where $i \in \mathbb{I} = \{1, 2, \dots, I\}$
j	The index of the boxes with the type i corresponding to customer c in the container k , where $j \in \mathbb{J} = \{1, 2, \dots, n_{ci}^k\}$
k	The index of containers, where $k \in \mathbb{K} = \{1, 2, \dots, K\}$
a	The index of container types, where $a \in \mathbb{A} = \{1, 2, \dots, A\}$
c	The index of customers, where $c \in \mathbb{C} = \{1, 2, \dots, C\}$
(L_k, W_k, H_k)	The length, width, and height of the container k

M_k	The maximum capacity of the container k
b_{ci}	The box of the type i corresponding to the customer c
n_{ci}	The quantity of b_{ci} in the whole loading task
b_{cij}	The j th box with the type i corresponding to the customer c
p_{cij}	The loading priority of b_{cij}
m_{cij}	The mass of b_{cij}
$(l_{cij}^k, w_{cij}^k, h_{cij}^k)$	The length, width, and height of b_{cij} loaded in the container k
$(\Delta x_{cij}^k, \Delta y_{cij}^k, \Delta z_{cij}^k)$	The length of the edge $O'A'$, $O'B'$, and $O'D'$ when b_{cij} is well placed in the container k
$(x_{cij}^k, y_{cij}^k, z_{cij}^k)$	The coordinate of the point O' when b_{cij} is well placed in the container k
g_{cij}^k	The variable that whether b_{cij} is loaded in container k
$u_{kcij}^{c'j'j'}$, $f_{kcij}^{c'j'j'}$, $r_{kcij}^{c'j'j'}$	The variables indicating the relative position between b_{cij} and $b_{c'j'j'}$
$d1_{cij}^k, d2_{cij}^k, d3_{cij}^k, d4_{cij}^k, d5_{cij}^k, d6_{cij}^k$	The variables indicating the placement direction of b_{cij}

2.2. Problem description

Consider a set $\mathbb{C}$ of customers, $\mathbb{C} = \{1, 2, \dots, C\}$, where 1 is the first customer to serve and C is the last. Each customer orders a set $\mathbb{I}$ of box types with different three-dimensional sizes, and every box type has a set $\mathbb{J}$ of boxes. Each type of box b_{ci} is associated with three dimensions: length, width, and height, denoted with (l_{ci}, w_{ci}, h_{ci}) , and hence a volume $v_{ci} = l_{ci}w_{ci}h_{ci}$. Besides, each box type b_{ci} has its weight m_{ci} and quantity n_{ci} . To contain the customers' orders, the company prepares a set of different types of containers, and the maximum capacity of the container k is represented by M_k .

A Cartesian coordinate system is established to describe the spatial relationship between the container and boxes. As shown in Fig. 1. The x -axis, y -axis, and z -axis of the coordinate system are used to quantify the length, width, and height of the container, respectively. The origin of the coordinate system is set as the intersection of the three planes. The box enters and exits from the front plane, and the loading starts from the lower left corner of the container. When the box is placed in the container, it can only be loaded orthogonally, that is, the box's edges are parallel to the container's edges as shown in Fig. 1.

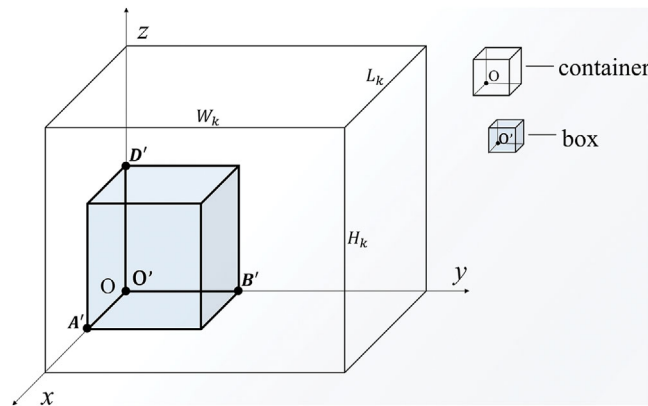


Fig. 1. The spatial relationship between the container and boxes.

Considering a loading task of packing nonidentical boxes using multiple containers, its 3D-MCLP is a weak heterogeneous container loading problem with goods following the LIFO constraints. The objective is to maximize the overall filling rate of multiple containers subject to several types of constraints:

1. *Weight limits*: There is a maximum capacity M_k that a container can carry.
2. *Stability*: Each box is placed on the floor of the container or completely supported by other boxes.
3. *Multidrop*: Each customer's boxes must be loaded in such a way that they can be unloaded without rearranging other customers' boxes.
4. *Combined loading*: The boxes of the same customer shall be placed adjacent to the container.
5. *Unloading constraint*: The unloading constraint is referred to as sequential constraint, they are relevant for multidrop constraint. In the paper, we make the unloading constraints to be a LIFO constraint.
6. *Orientation*: Each box is loaded in six possible ways based on its sizes on the x -axis, y -axis, and z -axis. However, some of these directions may not be allowed due to the limited available space. Each placement direction can be represented by the box's occupations on the three axes. Thus, b_{cij} has six possible placement directions in container k , that is, $(l_{cij}^k, w_{cij}^k, h_{cij}^k)$, $(l_{cij}^k, h_{cij}^k, w_{cij}^k)$, $(w_{cij}^k, h_{cij}^k, l_{cij}^k)$, $(w_{cij}^k, l_{cij}^k, h_{cij}^k)$, $(h_{cij}^k, l_{cij}^k, w_{cij}^k)$, and $(h_{cij}^k, w_{cij}^k, l_{cij}^k)$. The decision variables are defined as follows:

$$g_{cij}^k = \begin{cases} 1, & \text{if } b_{cij} \text{ is loaded in container } k \\ 0, & \text{otherwise} \end{cases}, \quad (1)$$

$$u_{kcij}^{c'ij'} = \begin{cases} 1, & \text{if } g_{cij}^k, g_{c'ij'}^k = 1, \text{ and } b_{c'ij'} \text{ is on the top of } b_{cij}, \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

$$f_{kcij}^{c'ij'} = \begin{cases} 1, & \text{if } g_{cij}^k, g_{c'ij'}^k = 1, \text{ and } b_{c'ij'} \text{ is in the front of } b_{cij}, \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

$$r_{kcij}^{c'ij'} = \begin{cases} 1, & \text{if } g_{cij}^k, g_{c'ij'}^k = 1, \text{ and } b_{c'ij'} \text{ is on the right side of } b_{cij}, \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

$$d1_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } l_{cij}^k \text{ and } w_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively,} \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

$$d2_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } l_{cij}^k \text{ and } h_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively,} \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

$$d3_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } w_{cij}^k \text{ and } l_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively,} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

$$d4_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } w_{cij}^k \text{ and } h_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively,} \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

$$d5_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } h_{cij}^k \text{ and } l_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively,} \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

$$d6_{cij}^k = \begin{cases} 1, & \text{if } g_{cij}^k = 1, \text{ and } h_{cij}^k \text{ and } w_{cij}^k \text{ are loaded along } z - \text{axis and } x - \text{axis, respectively.} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

2.3. Mathematical model

The general multiple three-dimensional container loading model is then constructed. The objective of this model is to maximize the overall filling rate of containers.

$$\max R = \frac{1}{k} \sum_{k=1}^K \frac{\sum_{c=1}^C \sum_{i=1}^I \sum_{j=1}^{n_{ci}} l_{cij}^k w_{cij}^k h_{cij}^k g_{cij}^k}{V_k}, \quad (11)$$

s.t.

$$x_{cij}^k - x_{c'i'j'}^k + L_k f_{kcij}^{c'i'j'} \leq L_k - \Delta x_{cij}^k, \quad \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (12)$$

$$y_{cij}^k - y_{c'i'j'}^k + W_k f_{kcij}^{c'i'j'} \leq W_k - y_{cij}^k, \quad \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (13)$$

$$z_{cij}^k - z_{c'i'j'}^k + H_k u_{kcij}^{c'i'j'} \leq H_k - z_{cij}^k, \quad \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (14)$$

$$0 \leq x_{cij}^k \leq L_k - x_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (15)$$

$$0 \leq y_{cij}^k \leq W_k - y_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (16)$$

$$0 \leq z_{cij}^k \leq H_k - z_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (17)$$

$$\sum_{c=1}^C \sum_{i=1}^I \sum_{j=1}^{n_{ci}^k} m_{cij}^k g_{cij}^k \leq M_k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (18)$$

$$\Delta x_{cij}^k = (d1_{cij}^k + d6_{cij}^k) w_{cij}^k + (d2_{cij}^k + d4_{cij}^k) h_{cij}^k + (d3_{cij}^k + d5_{cij}^k) l_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (19)$$

$$\Delta y_{cij}^k = (d1_{cij}^k + d3_{cij}^k) h_{cij}^k + (d2_{cij}^k + d5_{cij}^k) w_{cij}^k + (d4_{cij}^k + d6_{cij}^k) l_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (20)$$

$$\Delta z_{cij}^k = (d1_{cij}^k + d2_{cij}^k) l_{cij}^k + (d3_{cij}^k + d4_{cij}^k) w_{cij}^k + (d5_{cij}^k + d6_{cij}^k) h_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C}, \quad (21)$$

$$g_{cij}^k = d1_{cij}^k + d2_{cij}^k + d3_{cij}^k + d4_{cij}^k + d5_{cij}^k + d6_{cij}^k, \quad \forall k \in \mathbb{K}, i \in \mathbb{I}, j \in \mathbb{J}, c \in \mathbb{C} \quad (22)$$

$$u_{kcij}^{c'i'j'} + u_{kc'i'j'}^{cij} + f_{kcij}^{c'i'j'} + f_{kc'i'j'}^{cij} + r_{kcij}^{i'i'j'} + r_{kc'i'j'}^{cij} \geq g_{cij}^k + g_{c'i'j'}^k - 1, \quad \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (23)$$

$$\Delta x_{cij}^k u_{kij}^{k' i' j'} \geq \Delta x_{c' i' j'}^k \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (24)$$

$$\Delta y_{cij}^k u_{kij}^{k' i' j'} \geq \Delta y_{c' i' j'}^k \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (25)$$

$$u_{kij}^{c' i' j'} (p_{c' i' j'} - p_{cij}) \leq 0, \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}, \quad (26)$$

$$f_{kij}^{c' i' j'} (p_{c' i' j'} - p_{cij}) \leq 0, \forall k \in \mathbb{K}, i, i' \in \mathbb{I}, j, j' \in \mathbb{J}, c, c' \in \mathbb{C}. \quad (27)$$

Constraints (12)–(14) indicate the nonoverlapping between $b_{c' i' j'}$ and b_{cij} ; Constraints (15)–(17) indicate the placement of b_{cij} shall not exceed the length, width, and height of the container; Constraint (18) means that the weight of boxes does not exceed the allowed load of the container; Constraints (19)–(21) represent the length, width, and height of b_{cij} after its loading, respectively; Constraint (22) indicates that only one of the six possible placement plans can be used if b_{cij} is placed in container k ; Constraint (23) indicates that if b_{cij} and $b_{c' i' j'}$ are placed in container k , one of the relative positions between them, such as top, bottom, left, right, front and rear, must be true; Constraints (24) and (25) ensure that every box is completely supported: If $b_{c' i' j'}$ is placed on the top of b_{cij} , the top area of $b_{cij} \geq$ the top area of $b_{c' i' j'}$; Constraints (26) and (27) enforce the multidrop and LIFO constraint. By (26), if $b_{c' i' j'}$ is placed on the top of b_{cij} , $p_{c' i' j'} \leq p_{cij}$; By (27), if $b_{c' i' j'}$ is placed in the front of b_{cij} , the loading priority of the box $b_{c' i' j'} \leq$ that of b_{cij} .

3. Single container loading algorithm based on greedy heuristic multilayer tree search

The basis for solving a 3D-MCLP is finding an initial solution for each container. This section presents a single container loading algorithm based on greedy heuristic multilayer tree search.

As shown in Fig. 1a, one box loading plan can be described using the reference point O' , and the length of $O'A'$, $O'B'$, and $O'D'$. The information of boxes is recorded in the dataframe **BoxList**, where its data structure consists of $(l_{ci}, w_{ci}, h_{ci}, l_z, w_z, h_z, n_{ci}, m_{ci}, c, i, p)$. For each record, l_{ci} , w_{ci} , and h_{ci} indicate the length of the box edges. l_z , w_z , and h_z specify whether the edge with the length l_{ci} , w_{ci} , and h_{ci} can be placed vertically, respectively. For example, if l_z is 1, it means that the l_{ci} -long edge can be placed vertically; when it is 0, the box cannot be placed vertically. The dataframe **BoxList** is sorted by the following rules:

1. Sorted by the volume of the box in descending order.
2. Sorted by the loading priority value in descending order.

3.1. Block generation algorithm

The block-building approach was first proposed by Eley (2002). The block consists of stacked boxes of the same size in the same orientation corresponding to the same customer. As shown in Fig. 2, l_{ci} , w_{ci} , and h_{ci} represent the dimensions of the box on the x -axis, y -axis, and z -axis, respectively. b_l ,

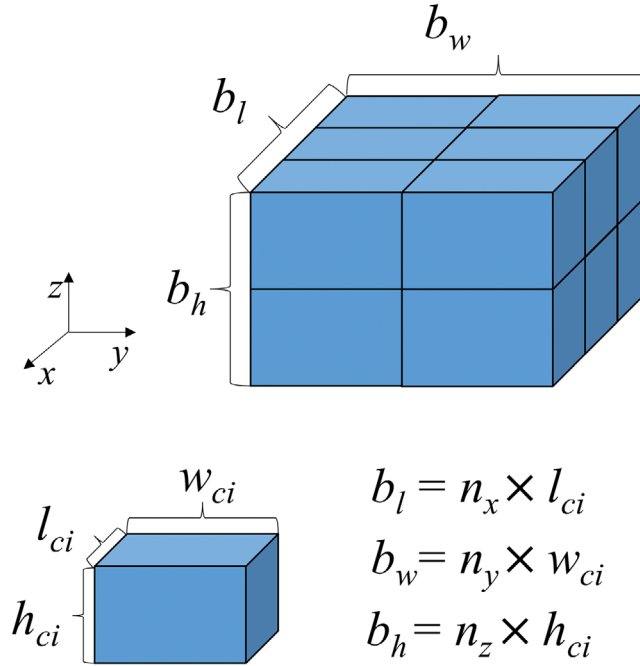


Fig. 2. Illustration of the block composition.

b_w , and b_h represent the length, width, and height of the block, respectively. n_x , n_y , and n_z denote the number of boxes in the block along the x -axis, y -axis, and z -axis, respectively.

$$n_x n_y n_z \leq n_{ci}, \quad (28)$$

$$n_x l_{ci} \leq L_x, \quad (29)$$

$$n_x w_{ci} \leq W_x, \quad (30)$$

$$n_z h_{ci} \leq H_x. \quad (31)$$

The boxes can be stacked into a block when the Constraints (28)–(31) are satisfied. Enumerate block combinations $(b_l, b_w, b_h, n, c, i, p)$ that meet the constraints, where $n = n_x \times n_y \times n_z$ indicates the number of boxes required for stacking into the block. It is worth noting that the values of n_x , n_y , and n_z are not fixed, which brings more possibilities for block generation. Then, add the block data to **BlockList**. Please see Policy 1 in the Appendix for details. The dataframe **BlockList** is sorted by the following order rules:

1. Sorted by b_h , b_w , and b_l in ascending order;
2. Sorted by the block volume $(b_l \times b_w \times b_h)$ in descending order;
3. Sorted by the loading priority value p in descending order.

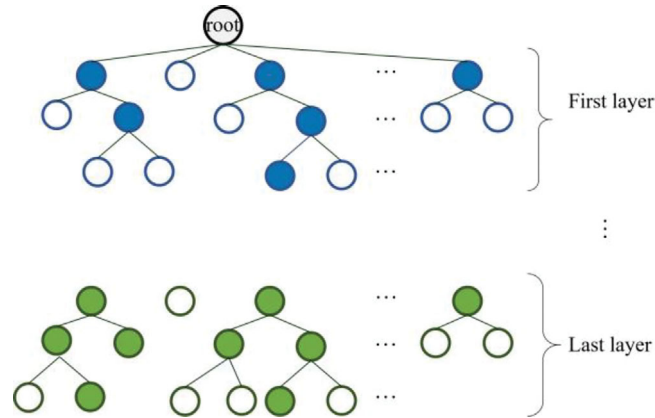


Fig. 3. The multilayer tree searches when $SubHeap = 2$.

3.2. Greedy heuristic multilayer tree search algorithm

Considering the LIFO rule, the loading sequence is determined according to the transportation distance of the customer. The loading priority of the customer's boxes with a long distance is higher than that of the customers with a short distance. This section uses a layer to describe a loading priority, that is, a customer's boxes are loaded by executing a layer's process. One layer operation includes multiple sublayer tree searches, each sublayer extends *RootHeap* nodes at most, and each node develops *SubHeap* branches at most. Figure 3 shows a tree search diagram with $SubHeap = 2$, where root represents the empty container, and the number of nodes of each sublayer represents the number of blocks that can be placed in the current residual space. The optimal blocks in the former *RootHeap* are selected to be placed, corresponding to the dark nodes of each sublayer in the figure. If the remained space of the container allows to place blocks, continue to search, and expand branch nodes.

The parent node loading information is recorded in *SearchTree*, where, *SearchTree* $\{1, i\}$ records the residual space set *ResidualSpace* of the i th node generated by the block loading algorithm. *SearchTree* $\{2, i\}$ records the loaded blocks and block reference point coordinates. *SearchTree* $\{3, i\}$ records the unloaded block data and removes invalid block data from the block list using the block update algorithm. The loading data of the child node is recorded in *Child*, whose structure is the same as *SearchTree*. Please find the Algorithm 2 in Appendix A.2 for details.

3.3. Block loading algorithm

The block loading algorithm expects to place a block in the current residual space *curSpace* and generates three new residual spaces. The initial residual space is the container. Three new residual spaces, referred to as *newSpace* (*nSpaceX*, *nSpaceY*, *nSpaceZ*), are generated by virtual cutting after placing a block. As shown in Fig. 4, the data in *nSpaceY* are structured with $(n_{sx}, n_{sy}, n_{sz}, r_{ex}, r_{ey}, r_{ez}, s_{xyz})$, where n_{sx} , n_{sy} , and n_{sz} mean the length, width, and height of the newly added residual space *nSpaceY*; r_{ex} , r_{ey} , and r_{ez} represent the left rear lower corner coordinate of *nSpaceY*; and

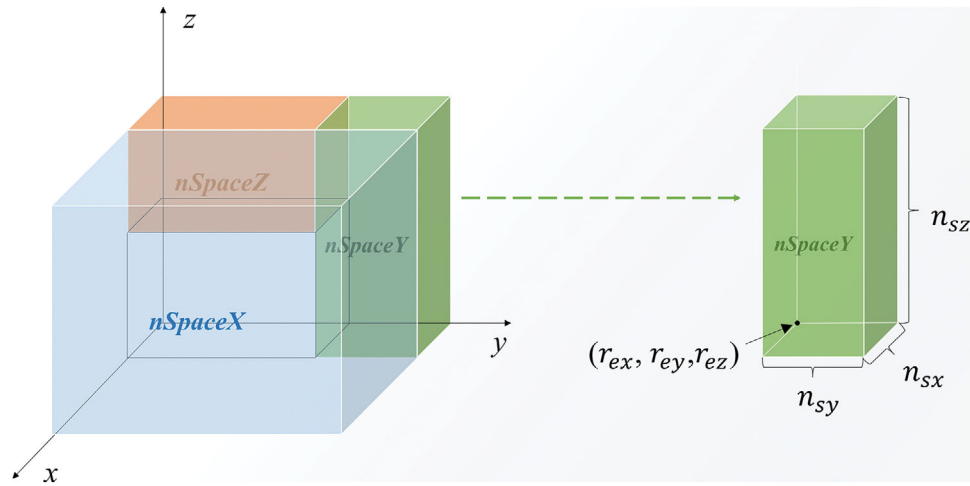
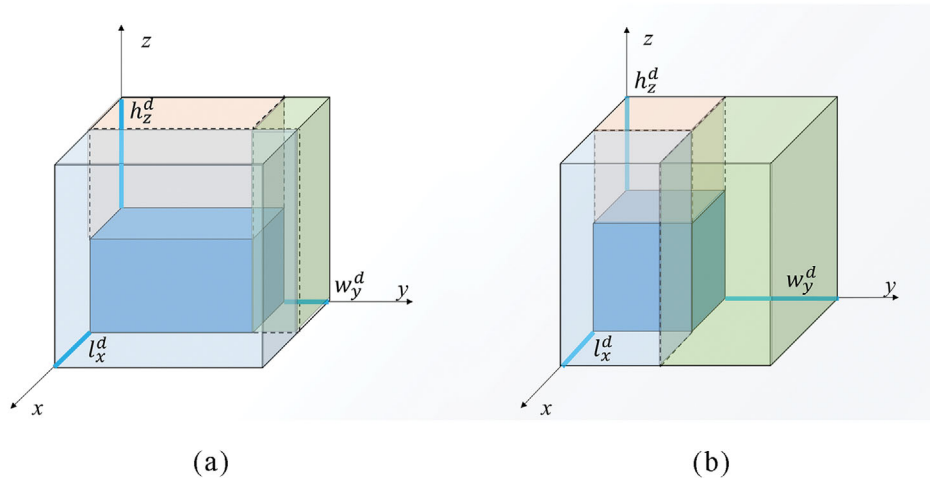


Fig. 4. Illustration of the residual space.

Fig. 5. Possible cutting plans of the residual space. (a) Cutting plan when $l_x^d \geq w_y^d$ and (b) Cutting plan when $l_x^d \leq w_y^d$.

$s_{xyz} = \{1, 2, 3\}$ indicates the cutting way. The elements in s_{xyz} , that is, 1, 2, 3, represent cutting along the x -axis, y -axis, and z -axis and generating the residual space $nSpaceX$, $nSpaceY$, $nSpaceZ$, respectively.

The pseudocode of the block loading algorithm is shown in Appendix A.3 After the block is placed in the residual space *curSpace*, l_x^d , w_y^d , and h_z^d denote the residual lengths of the x -axis, y -axis, and z -axis, respectively. To make the residual space cut as large as possible, cut the residual space according to the relationship between l_x^d and w_y^d . If $l_x^d \geq w_y^d$, cut the residual space according to Fig. 5a; otherwise, cut the residual space according to Fig. 5b. The reference point of *curSpace* indicates the reference point coordinates of the block placed.

3.4. Block selection algorithm

Based on the greedy idea, the block loading algorithm heuristically searches the block index set and the residual space *curSpace* in *ResidualSpace*. To place a block in *curSpace*, the following three conditions must be met:

Condition 1. The size of the block meets the placement constraint, that is, $b_l \leq n_{sx}$, $b_w \leq n_{sy}$, $b_h \leq n_{sz}$.

Condition 2. The newly generated residual space is available or the volume of the unavailable residual space is less than a given small volume *minvolume*. The three newly generated residual spaces shall meet Condition 2. Take the residual space *nSpaceY* in Fig. 4 as an example, if $\min(\text{BlockList}(:, 2)) \leq n_{sz}$, that is, *nSpaceY* is available, blocks can be placed in *nSpaceY*; otherwise, the condition is satisfied when *nSpaceY* is smaller than *minvolume*. For example, given a container with three dimensions [587, 233, 220], *curSpace*(1:3) = [456, 99, 220], the length, width, and height of the placed block are 445, 87, and 216, respectively. In this case, the volume of *nSpaceY* is 1203840 ($456 \times 12 \times 220$) by cutting in the way of Fig. 5b and occupies 4% of the total volume of the container; thus, *nSpaceY* is not available, and it will cause a waste of container space if the block is placed in *curSpace*. The transition block index set that meets Conditions 1 and 2 is output using Algorithm 4 in Appendix A.4.

Condition 3. Customer orders with high loading priority are loaded first, that is, the larger the *p*-value, the smaller the value of the corresponding layer, and the priority of the block loaded at the layer level is $p = \text{customers}_{types} - \text{layer} + 1$. It is prohibited to place the blocks with high loading priority in the top or front areas of the blocks with low priority. Region (1,1:3) stores the left rear lower corner coordinate of the top space of the block, Region (1,4:6) stores the right front upper corner coordinate of the top space, and Region (2,1:6) records the front space of the block.

The block selection algorithm searches for block indexes that meet the above three conditions. Please see Algorithm 5 in Appendix A.5 for details.

3.5. Block update algorithm

Invalid blocks must be identified and processed to ensure consistency in block and box loading rates and reduce the algorithm's search space. The block update algorithm removes invalid block data from the *Blocklist*. One block will be seen as invalid if either of the following two conditions is satisfied:

Condition 1. The number of boxes to form the block exceeds the remaining available boxes.

Condition 2. The volume of the block is greater than the maximum residual space volume in *ResidualSpace*.

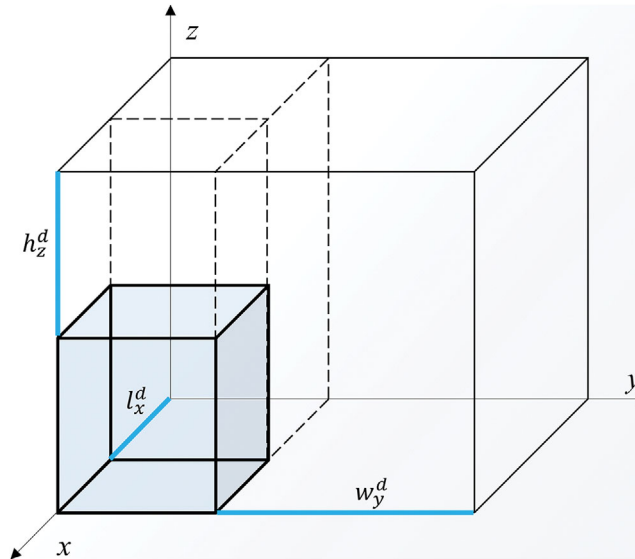


Fig. 6. Illustration for cutting the residual space after loading a complex box in the *ProBoxList*.

3.6. Other operations

3.6.1 Complex box identifying

A complex box means a box with extremely long edges or a large volume. Given some complex boxes may belong to the customer with low priority, they could not be loaded or partially loaded due to the principle of LIFO. Thus, the complex box identifying algorithm is proposed to find out whether there are complex boxes with low loading priority. First, calculate the total volume of all boxes in *BoxList*. Then, cumulate the volume of the ordered boxes and add the corresponding records to the low loading priority *Lowlp*, if its volume is greater than $0.5 \times \text{total volume}$. If one box edge in *Lowlp* $\geq$ the complex edge factor or the box volume $\geq$ complex volume factor, the box is recognized as a complex box and added to *ProBoxList*. (See Algorithm 6 in the Appendix for details.)

3.6.2 Block loading with complex boxes

After removing *ProBoxList* from *BoxList*, the remaining part is counted as *AfterBoxList*. The starting loading points of *ProBoxList* and *AfterBoxList* are different. The initial loading point of *ProBoxList* is located at the rear door of the container. Therefore, the data in *ProBoxList* are sorted in ascending order according to the loading priority, so that boxes with low loading priority can be placed near the door of the container. A greedy heuristic multilayer tree search algorithm is used to load *ProBoxList* and *AfterBoxList*, sequentially. The block loading algorithm in Section 3.3 is used for cutting to generate new residual spaces when loading *AfterBoxList*, while the complex block loading algorithm is used for cutting when loading *ProBoxList* as shown in Fig. 6. (See Algorithm 7 in the Appendix for details.)

3.6.3 Block selection algorithm with complex boxes

Complex boxes with low loading priority are preferentially loaded at the container door. The bottom region restriction of blocks should be added to the block selection algorithm to satisfy the LIFO constraint. Region $(3,1:6) = [\text{curSpace}(4), \text{curSpace}(5), 0, \text{curSpace}(1:2) + \text{curSpace}(4:5), \text{curSpace}(6)]$. If the loaded block has an intersection with the bottom of the current loaded block, compare the priority between the previous loaded block and the current loaded block. The LIFO constraint is satisfied only when the former is greater than or equal to the latter.

4. Multiple containers loading algorithm integrating a large neighborhood heuristic search

To solve the problem of multiple container loading problem effectively, this section presents a large neighborhood heuristic search algorithm to allocate and load heterogeneous containers integrating the above greedy heuristic multilayer tree search method. Considering the LIFO constraint of a 3D-MCLP, novel removal and repair operators are designed.

4.1. Initial solution generation

Given the dataset of all boxes to be loaded and the available containers sequence, a greedy heuristic multilayer tree search algorithm proposed in Section 3 is used to search **BoxList** in turn and place it in the container. Although containers are weak and heterogeneous in this research, the container with the highest filling rate is selected for loading when the single container loading operation is performed. Output the initial solution until all boxes are loaded in the containers, or all containers are loaded. The operation flow chart is shown in Fig. 7.

4.2. Removal operators

The removal operation removes all boxes loaded corresponding to several customers in one iteration, leaving the remaining customers' boxes unchanged. The specific removal operators are introduced as follows:

1. *Random removal operator*: This randomly removes q customers in the current solution.
2. *Worst filling rate removal operator*: This removes the boxes of the customers with the lowest filling rate. For example, if the box of customer c is placed in the container with the lowest filling rate, remove all boxes of customer c , including those loaded in other containers.
3. *Largest waste space removal operator*: This removes the boxes of the customers in the container with the largest waste space. Sometimes the filling rate of a container is high, much space is still wasted due to the container's large volume. This removal operation aims to reduce the wasted space.
4. *Customer distance-oriented removal operator*: This selects a customer c randomly from the current solution to remove and then removes $q - 1$ customer with a similar distance to customer c . Customer distance similarity is illustrated by the priority difference between customers, that is,

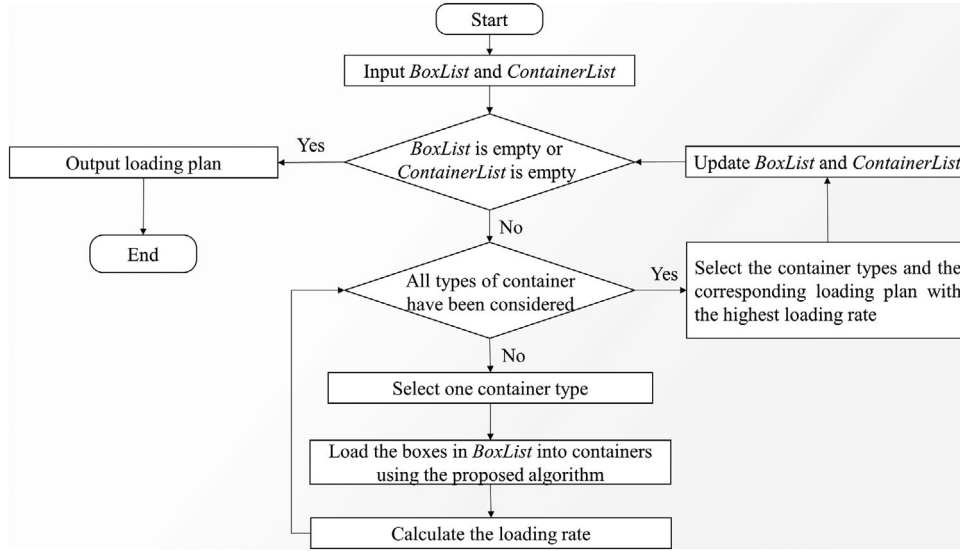


Fig. 7. Flow chart of initial solution generation for a 3D-MCLP.

$\Delta p_{c_1 c_2} = |p_{c_1} - p_{c_2}|$, where p_{c_1} and p_{c_2} represent the loading priority of customers c_1 and c_2 , respectively.

5. *Customer distribution-oriented removal operator*: This removes customers whose orders are loaded in more than d containers in the current solution, aiming to mostly load the same customer boxes in the same container.

Please note that both q and d are constants where q depends on the number of loaded customers n_c . Given a certain percentage α , $q = \lceil \alpha n_c \rceil$. d is a small value, which is not more than 3 in this research.

4.3. Repair operators

The repair operation will reload the removed boxes into the container and use the roulette method to select the operator; the repair operators are shown as follows.

1. *Greedy repair operator*: The removed customers' boxes will be loaded into the container with the highest filling rate based on the greedy idea. This approach is shortsighted because it only considers the single container with the highest filling rate.
2. *Regret repair operator*: The highest filling rate of a single container does not mean that the overall filling rate of multiple containers is the highest. The removed customers' boxes are loaded into the container with the second highest filling rate.

In practical application, we obtain the possibility of each removal operator and repair operator through pre-experiment. After the initial solution is generated, it is iteratively improved by removing and repairing operators until the termination condition is reached.

5. Numerical experiments

Numerical experiments of the proposed loading algorithm are completed by MATLAB R2018b software and run using a personal computer (Intel® Core™ i5-7500 CPU @ 3.40 GHz). Considering different sizes of available containers, set *minvolume*, *complexedge*, and *complexvolume* according to certain percentages λ , μ , and ν , respectively. Where $\text{minvolume} = \lambda L_k W_k H_k$, $\text{complexedge} = \mu * \min\{L_k, W_k, H_k\}$, $\text{complexvolume} = \nu L_k W_k H_k$. *RootHeap* = 10, *SubHeap* = 1, $\lambda = 0.01$, $\mu = 0.5$, $\nu = 0.02$, $\alpha = 0.2$, $d = 3$, and the maximum number of iterations is 50.

This section uses the multiple container loading problems, that is, Case A, Case B, and Case X, introduced by Renault for the 2014/2015 ESICUP (EURO Special Interest Group on Cutting and Packing) challenge to validate the proposed initial solution generation method (Toffolo et al., 2017). Then, the real-world company loading data are used to further test the performance of the proposed 3D-MCLP approach.

5.1. Results of the 2014/2015 ESICUP challenge cases

Since customers are not mentioned in the challenge cases, only the proposed initial solution method is applied. The results are compared with those computed by the two-dimensional heuristic decomposition method (TDHD) proposed by Toffolo et al. (2017). To improve the quality of the initial solution, set the parameter *SubHeap* = 2 for the challenge case and leave the other parameters unchanged. (See Tables 1–3 for the detailed results.), where Obj_2 is the volume of goods in the smallest container, Obj_1 is the total volume of all containers with loaded goods except Obj_2 , and the goal is to minimize Obj_1 . Please note that * in Tables 1–3 means the difference between the results of the proposed approach and TDHD approach, and $\text{gap}\%$ is obtained using the equation $(\text{Obj}_1 \text{ of the proposed approach} - \text{Obj}_1 \text{ of the TDHD approach}) / \text{Obj}_1 \text{ of the TDHD approach}$.

It can be seen from Tables 1–3 that the proposed approach can calculate the optimal solution with the running time reduced by 97%. The difference between the results of the two methods is below 10%. In some tests, the results using the proposed method are even better than the latter, which are marked in bold in the tables. Most of the 19 test problems of Case B in Table 2 are loaded with one type of container, and the performance of the proposed approach in Case B is significantly better than that in Cases A and X. Thus, adding the loading priority constraint outputs better results when addressing most loading problems with one type of a container.

In order to comprehensively compare the performance of the two algorithms, we plot the performance profile (Dolan and Moré, 2002) on Case A with a loading rate as the performance metric in Fig. 8. As the figure shows, the TDHD algorithm can only solve 70% of the subset of Case A, which is comparable to the initial solution of the proposed algorithm, and it takes much more time than ours. It should be pointed out that three out of the five removal operators designed by the proposed algorithm are related to customers, and the data of customers are not mentioned in the challenge cases (Cases A, B, and X), so we cannot compare the performance of the two algorithms by plotting the time-to-target (TTT) graph (Aiex et al. 2007).

Table 1
Results of Case A.

Test	Lower bound		Two-dimensional heuristic decomposition approach			The proposed approach			
	Obj ₁	Obj ₂	Obj ₁	Obj ₂	Running time/s	Obj ₁	Obj ₂	Running time/s	Gap%*
ARG BUR STK	228.43	30.85	275.18	25.35	654.50	294.78	16.86	17.55	7.12
AVF RIR STK	1111.62	32.26	1312.19	24.47	1542.90	1378.16	9.70	23.99	5.03
AVF RIR STK	353.12	69.90	441.40	50.86	882.90	447.94	59.19	21.69	1.48
AVT BUR STK	152.29	33.37	208.84	15.32	562.20	228.43	17.29	5.81	9.38
AVZ RIR STK	548.23	31.91	644.51	22.86	1154.60	680.16	23.51	20.30	5.53
AVZ RIR STK	353.12	29.50	441.40	33.09	797.60	447.94	32.15	20.33	1.48
CHE GC PEX	206.81	25.83	224.64	40.13	634.80	224.64	41.25	9.34	0.00
CHE RIR STK	371.73	28.27	470.68	15.75	919.90	497.42	2.08	18.99	5.68
CME GC PEX	239.80	26.99	281.69	24.81	687.70	290.61	22.98	9.98	3.17
CVP GC PEX	973.45	55.20	1231.96	21.25	1432.50	1171.35	19.09	33.78	-4.92
CVU GC PEX	4614.06	32.44	5255.01	23.76	4564.80	5575.04	5.60	50.06	6.09
FSI GC PEX	1198.09	57.97	1375.48	17.47	1719.60	1486.02	54.26	28.77	8.04
IND RIR STK	206.81	32.22	248.71	23.16	631.60	257.62	8.28	9.61	3.58
MED GC PEX	449.28	64.23	587.45	9.25	860.60	623.11	5.80	14.58	6.07
NSA BUR STK	152.29	41.34	208.84	21.71	545.90	209.03	13.49	3.85	0.09
NSA RIR STK	149.76	58.41	224.64	20.13	611.10	215.73	28.18	7.62	-3.97
RSM GC PEX	332.51	31.12	374.40	36.15	659.10	407.39	11.98	3.14	8.81
SOM BUR STK	152.29	26.84	185.60	26.80	561.70	185.60	6.54	14.68	0.00
SOM RIR STK	1048.33	33.66	1210.57	20.04	1515.10	1309.52	14.39	29.20	8.17
TNG RIR STK	215.73	26.30	272.78	17.69	639.70	290.61	6.35	4.04	6.54
Average results	652.89	38.43	773.80	24.50	1078.94	811.06	19.95	17.37	3.87

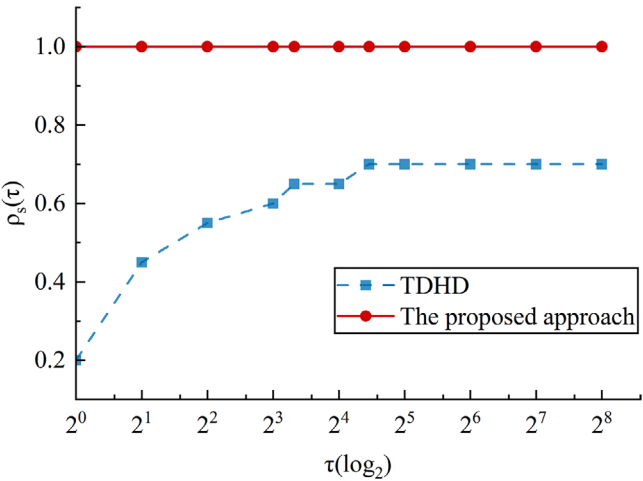


Fig. 8. Performance profile on Case A for two algorithms.

Table 2
Results of Case B.

Test	Lower bound		Two-dimensional heuristic decomposition approach			The proposed approach			
	Obj ₁	Obj ₂	Obj ₁	Obj ₂	Running time/s	Obj ₁	Obj ₂	Running time/s	Gap%*
ACI RIR PEX	353.12	1.72	353.12	1.78	565.50	353.12	4.29	8.09	0.00
ALG RIR PEX	1797.13	40.06	2021.78	10.16	2274.40	2021.78	52.29	33.65	0.00
AVF RIR PEX	374.40	43.99	449.28	22.01	832.80	449.28	44.49	7.20	0.00
AVZ RIR PEX	673.93	36.17	823.69	46.80	1159.80	823.69	49.39	19.97	0.00
CHE GC PEX	275.35	61.05	342.17	32.46	769.50	332.52	33.41	8.69	-2.82
CHE RIR PEX	673.93	49.30	823.69	12.66	1284.50	823.69	22.83	16.42	0.00
CHE VLD PEX	599.04	20.90	673.93	27.17	867.50	673.93	20.21	7.40	0.00
CVP GC PEX	449.28	45.63	524.16	38.16	928.90	524.18	35.84	12.36	0.00
CVU GC PEX	1422.73	51.34	1572.49	49.77	1730.20	1722.25	28.48	20.67	9.52
CVU VLD PEX	449.28	59.29	524.16	33.65	990.30	524.18	9.53	8.22	0.00
FSI GC PEX	599.04	66.00	673.93	59.67	1133.40	748.82	19.52	18.71	11.11
FSI VLD PEX	898.57	52.11	1198.09	34.40	1353.90	1272.97	11.79	31.04	6.25
IKO RIR PEX	353.12	33.80	372.73	77.82	825.90	379.27	32.26	15.02	1.76
MED GC PEX	449.28	58.23	599.04	20.41	860.60	599.04	9.30	5.91	0.00
MJV VLD PEX	264.84	61.72	353.12	31.65	664.60	379.27	1.21	6.49	7.41
NSA RIR PEX	1198.09	38.62	1347.85	31.86	1557.20	1422.73	40.75	17.53	5.56
RSM GC PEX	449.28	4.42	524.16	28.02	779.70	449.28	46.18	7.20	-14.29
SOM VLD PEX	2471.06	2.32	3444.51	20.12	2371.70	2845.46	32.05	32.59	-17.39
TAK RIR PEX	176.56	30.00	176.56	54.29	628.80	183.10	46.88	8.97	3.70
Average results	733.05	39.82	884.13	33.31	1135.75	869.92	28.46	15.06	0.57

5.2. Results of real-world cases

A real-world company uses two types of containers for product shipment. It faces the problems that some boxes cannot be loaded, the container filling rate is low, and some boxes are repeatedly taken out and reloaded. Please find the loading task information in Table 4.

Table 5 records the loading results of the real-world case studies with the proposed approach, where the optimal results output by the initial solution and the improved solution are compared. The average filling rate of the initial solution is about 85%, which has exceeded the manual daily filling rate of the enterprise, and the calculation time is also acceptable. The improved solution takes more time than the initial solution but has higher volume utilization, with an average filling rate of 89.47% and an average increase of 5.26%. Note that * in Table 5 means that Increased filling rate = (filling rate of the improved solution - filling rate of the initial solution) / filling rate of the initial solution.

It can be seen that case 8 shows the greatest improvement with an increase of 7.71%. The company uses 950*245*260 cm³ and 760*245*255 cm³ containers to assemble boxes, which are represented by Type 1 and Type 2, respectively. See Table 6 for the container types and filling rates used in the initial and improved solutions of test case 8, where the filling rates are sorted in descending order. As can be seen from panel a of Table 6, the initial solution outputs the blocks that can be

Table 3
Results of Case X.

Test	Lower bound		Two-dimensional heuristic decomposition approach			The proposed approach			
	Obj ₁	Obj ₂	Obj ₁	Obj ₂	Running time/s	Obj ₁	Obj ₂	Running time/s	Gap%*
ALG RIR PEX	685.29	49.99	761.43	45.42	1057.60	808.18	38.07	32.55	6.14
ARG BUR PEX	913.71	59.37	1069.78	17.09	1316.00	1145.93	12.68	31.59	7.12
AVF RIR PEX	1066.00	57.98	1202.48	23.36	1476.00	1202.48	29.60	34.14	0.00
AVT BUR PEX	989.86	35.21	1132.35	24.22	1236.30	1222.07	17.34	3.56	7.92
BAX COR PEX	821.76	29.56	983.85	21.02	1144.70	1030.60	9.33	24.16	4.75
CHE RIR PEX	523.20	31.03	609.14	43.85	974.10	622.72	29.04	30.73	2.23
CME GC EXP	341.53	30.37	380.71	43.68	790.20	413.89	34.09	23.19	8.71
CVP GC EXP	989.86	53.29	1219.84	25.29	1276.40	1188.90	10.69	30.84	-2.54
FSI GC EXP	1725.68	32.54	1901.34	12.86	1960.60	2049.84	36.57	39.74	7.81
MED GC EXP	911.49	31.18	1073.57	21.08	1202.90	1093.16	10.39	26.78	1.82
NSA RIR PEX	456.86	68.23	566.17	30.01	858.10	589.55	19.67	15.96	4.13
RSM GC EXP	284.98	30.89	361.12	14.01	661.20	370.92	2.74	46.93	2.71
SOM GC EXP	685.29	61.71	811.97	22.80	1070.60	864.73	14.54	9.11	6.50
SOM RIR PEX	1522.86	66.64	1871.96	26.39	1740.90	1887.76	5.72	34.84	0.84
TAN BUR PEX	533.00	40.77	646.10	18.81	991.70	689.07	23.26	10.23	6.65
TNG RIR PEX	2937.29	32.89	3232.73	9.50	3193.10	3365.42	2.63	30.34	4.10
VAX COR PEX	228.43	36.75	304.57	21.76	629.30	308.36	13.49	9.07	1.24
Average results	918.65	44.02	1066.42	24.77	1269.39	1109.03	18.23	25.52	4.13

Table 4
A real-world loading task

Test case	Number of container types	Number of customers	Number of box types	Number of boxes
1	2	5	27	610
2	2	10	52	1400
3	2	15	79	1960
4	2	20	113	2700
5	2	25	140	3430
6	2	30	179	4240
7	2	35	211	5040
8	2	40	237	5595
9	2	45	264	6205
10	2	50	285	6745

placed in order, resulting in a lower filling rate of the last few containers. The worst filling rate removal operator and the worst wasted space removal operator in the improved solution remove all customer boxes in these containers and re-heuristic loading based on a large neighborhood heuristic search. This operation effectively improves the total filling rate of the containers. It can be seen from the two tables that both the initial solution and the improved solution loaded all the boxes. The initial plan uses 21 one-containers and 16 two-containers, while the improved scheme uses 17

Table 5
Results output by the initial solution and improved solution

Real-world case	Initial solution		Improved solution		Increased filling rate %*
	Filling rate/%	Running time/s	Filling rate/%	Running time/s	
1	85.06	5.31	89.13	52.62	4.79
2	85.32	10.70	90.15	178.62	5.65
3	84.69	17.53	88.58	284.62	4.59
4	85.80	30.67	88.23	500.92	2.83
5	84.43	46.79	90.21	670.73	6.85
6	84.93	70.74	89.50	912.07	5.38
7	85.06	98.85	88.94	1105.63	4.57
8	85.21	117.19	91.78	1218.50	7.71
9	84.17	135.34	89.89	1370.71	6.79
10	85.26	180.35	88.25	1480.17	3.50
Average	84.99	71.35	89.47	777.46	5.26

Table 6
Containers filling rates of real-world case 8.

(a) Containers filling rates with the initial solution											
No.	Type	Filling rate/%	No.	Type	Filling rate/%	No.	Type	Filling rate/%	No.	Type	Filling rate/%
1	1	94.70	11	2	90.75	21	1	86.85	31	1	80.39
2	2	94.26	12	1	90.74	22	2	86.59	32	1	79.84
3	2	94.08	13	1	90.31	23	1	86.13	33	1	75.27
4	2	93.23	14	1	89.78	24	1	85.90	34	2	73.49
5	2	92.82	15	2	89.36	25	1	85.68	35	2	72.53
6	2	92.34	16	2	88.96	26	2	85.29	36	1	68.39
7	1	92.03	17	1	88.55	27	1	85.07	37	2	29.83
8	1	91.65	18	1	87.92	28	2	83.81			
9	1	91.47	19	2	87.86	29	1	83.53			
10	2	90.98	20	1	86.92	30	1	81.65			

(b) Containers filling rates with the improved solution											
No.	Type	Filling rate/%	No.	Type	Filling rate/%	No.	Type	Filling rate/%	No.	Type	Filling rate/%
1	2	95.72	10	1	93.49	19	1	91.47	28	1	90.35
2	2	94.69	11	1	93.43	20	1	91.44	29	2	89.95
3	2	94.57	12	2	93.08	21	1	91.10	30	2	89.86
4	1	94.37	13	1	92.64	22	1	90.92	31	1	89.71
5	1	93.99	14	2	92.09	23	2	90.77	32	1	89.49
6	2	93.94	15	1	91.96	24	2	90.75	33	2	89.42
7	1	93.75	16	2	91.89	25	1	90.59	34	2	88.45
8	2	93.62	17	2	91.74	26	2	90.57	35	2	86.39
9	1	93.57	18	2	91.51	27	1	90.40			

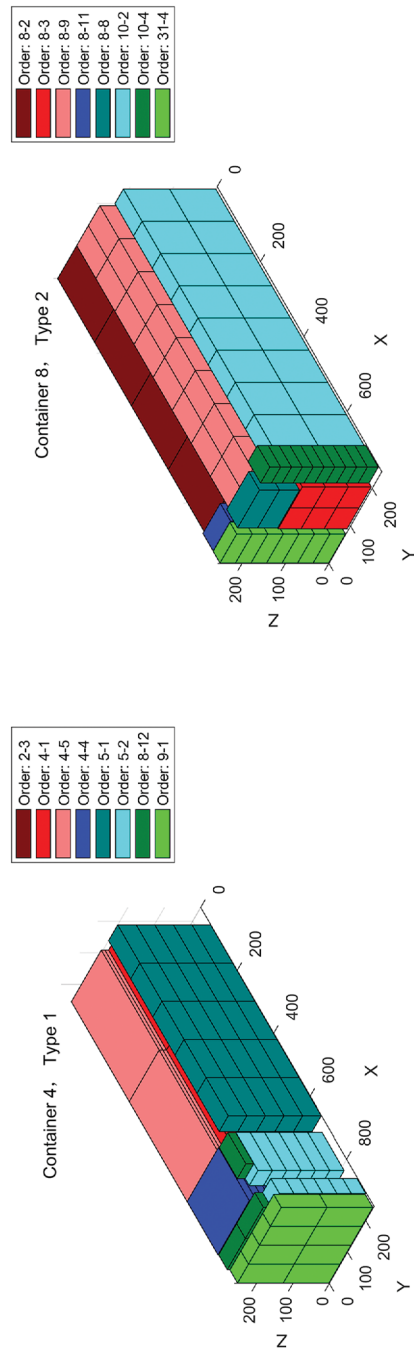


Fig. 9. Visualization of loading solutions for two containers.

one-containers and 18 two-containers, effectively reducing the total number of required containers. Thus, the total filling rate of multiple containers is improved.

Figure 9 visualizes the loading solution of two containers, which belong to the first and second types of containers, respectively. The legend illustrates the matching relationship between boxes and customers, for example, “order: 2–3” indicates that the order is the third type of box for customer 2.

6. Conclusion

In this paper, we studied the heterogeneous three-dimensional multiple containers loading problem with the LIFO constraint. We established an integer programming model with the objective function of maximizing the overall filling rate of multiple containers. A greedy heuristic multilayer tree search-based algorithm was proposed first to search the initial solution for a 3D-MCLP. Then, a large neighborhood heuristic search was integrated to effectively improve the solution quality. The numerical results proved the effectiveness and efficiency of the proposed approach when solving a general 3D-MCLP.

The contributions are mainly threefold.

1. The proposed 3D-MCLP approach realizes an average filling rate of around 90% in real-world loading cases, considering complex constraints: LIFO, weak heterogeneous containers, and various destinations.
2. As the loading information is well recorded in the dataset, the proposed container loading approach efficiently provides the company with optimal loading plans. The intelligent and fast decision process helps to increase loading efficiency and filling rate and reduce human and transportation costs.
3. The proposed approach increases the unloading efficiency by considering the LIFO constraints. The boxes corresponding to each customer are sequentially blocked and pushed into the container based on the transportation distance, realizing convenient unloading.

Due to the complexity of the loading problem and the increase in the order data size, there will be more combinations of blocks, and the search range will be larger and more complex when dealing with a 3D-MCLP. The proposed approach may require a long calculation time in such a scenario. Therefore, one direction of future work is to optimize the algorithm further and improve computational efficiency.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (Grant No. 52075354, 72201186), the Natural Science Foundation of Jiangsu Province (Grant No. BK20220481), the Natural Science Foundation of the Jiangsu Higher Education Institutions of China (Grant No. 22KJB410002), and the China Postdoctoral Science Foundation (Grant No. 2023M731450).

References

- Aiex, R.M., Mauricio G.C. Resende, M.G.C., Ribeiro, C.C., 2007. TTT plots: a Perl program to create time-to-target plots. *Optimization Letters*, 1, 355–366.
- Alonso, M.T., Alvarez-Valdes, R., Iori, M., Parreño, F., Tamarit, J.M., 2017. Mathematical models for multicontainer loading problems. *Omega* 66, 106–117.
- Alonso, M.T., Alvarez-Valdes, R., Parreño, F., 2020. Mathematical models for multicontainer loading problems. *Operation Research* 18, 4972.
- Bischoff, E.E., Janetz, F., Ratcliff, M.S.W., 1999. Loading pallets with non-identical items. *European Journal of Operational Research* 84, 681–692.
- Bischoff, E.E., Ratcliff, M.S.W., 1995. Issues in the development of approaches to container loading. *Omega* 23(4), 377–390.
- Bonet, F.G., Trivella, A., Corman, F., 2023. Modeling soft unloading constraints in the multi-drop container loading problem. *European Journal of Operational Research* 308, 336–352.
- Bortfeldt, A., Wäscher, G., 2013. Constraints in container loading—a state-of-the-art review. *European Journal of Operational Research* 229, 1–20.
- Chen, C., Lee, S.M., Shen, Q., 1995. An analytical model for the container loading problem. *European Journal of Operational Research* 80, 68–76.
- Christensen, S.G., Rousøe, D.M., 2009. Container loading with multi-drop constraints. *International Transactions in Operational Research* 16, 727–743.
- Dolan, E.D., Moré, J.J., 2022. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91, 201–213.
- Eley, M., 2002. Solving container loading problems by block arrangement. *European Journal of Operational Research* 141, 393–409.
- Fanslau, T., Bortfeldt, A., 2010. A tree search algorithm for solving the container loading problem. *INFORMS Journal of Computing* 22(2), 222–235.
- Gehring, H., Bortfeldt, A., 2002. A parallel genetic algorithm for solving the container loading problem. *International Transactions in Operational Research* 9(4), 497–511.
- Giménez-Palacios, I., Alonso, M.T., Alvarez-Valdés, R., Parreño, F., 2023. Multi-container loading problems with multidrop and split delivery conditions. *Computers and Industrial Engineering* 175, 108844.
- Gonçalves, J.F., Resende, M.G.C., 2013. A biased random key genetic algorithm for 2D and 3D bin packing problems. *International Journal of Production Economics* 145(2), 500–510.
- Hifi, M., Kacem, I., Nègre, S., 2010. A linear programming approach for the three-dimensional bin-packing problem. *Electronic Notes in Discrete Mathematics* 36, 993–1000.
- Huang, W.Q., He, K., 2009. A caving degree approach for the single container loading problem. *European Journal of Operational Research* 196, 93–101.
- Jamrus, T., Chien, C., 2016. Extended priority-based hybrid genetic algorithm for the less-than-container loading problem. *Computers and Industrial Engineering* 96, 227–236.
- Li, X., Zhang, K., 2015. A hybrid differential evolution algorithm for multiple container loading problem with heterogeneous containers. *Computers and Industrial Engineering* 90(DEC.), 305–313.
- Lim, A., Ma, H., Xu, J., 2012. An iterated construction approach with dynamic prioritization for solving the container loading problems. *Expert Systems with Applications* 39, 4292–4305.
- Lin, Y., Meller, R.D., Ellis, K.P., 2013. A decomposition-based approach for the selection of standardized modular containers. *International Journal of Production Research* 392, 6125–6134.
- Liu, J., Yue, Y., Dong, Z., Maple, C., Keech, M., 2011. A novel hybrid tabu search approach to container loading. *Computers and Operations Research* 38(4), 797–807.
- Liu, S., Shang, X., Cheng, C., Zhao, H., Shen, D., Wang, F., 2017. Heuristic algorithm for the container loading problem with multiple constraints. *Computers and Industrial Engineering* 108(Jun.), 149–164.
- Mack, D., Bortfeldt, A., Gehring, H., 2004. A parallel hybrid local search algorithm for the container loading problem. *International Transactions in Operational Research* 11(5), 511–533.

- Moura, A., Oliveira, J.F., 2005. A grasp approach to the container-loading problem. *IEEE Intelligent Systems* 20(4), 50–57.
- Paquay, C., Schyns, M., Limbourg, S., 2016. A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *International Transactions in Operational Research* 23, 187–213.
- Parreño, F., Alvarez-Valdés, R., Tamarit, J.M., Oliveira, J. F., 2008. A maximal-space algorithm for the container loading problem. *INFORMS Journal on Computing* 20(3), 412–422.
- Rajaei, M., Moslehi, G., Reisi-Nafchi, M., 2024. The multiple container loading problem with loading docks. *International Transactions in Operational Research* 31, 1671–1698.
- Tian, T., Zhu, W., Lim, A., Wei, L., 2016. The multiple container loading problem with preference. *European Journal of Operational Research* 248(1), 84–94.
- Toffolo, T.A.M., Esprit, E., Wauters, T., Vanden-Berghe, G., 2017. A two-dimensional heuristic decomposition approach to a three-dimensional multiple container loading problem. *European Journal of Operational Research* 257(2), 526–538.
- Wei, L., Zhang, Z., Lim, A., 2014. An adaptive variable neighborhood search for a heterogeneous fleet vehicle routing problem with three-dimensional loading constraints. *IEEE Computational Intelligence Magazine* 9(4), 18–30.
- Wu, Y., Li, W., Goh, M., de Souza, R., 2010. Three-dimensional bin packing problem with variable bin height. *Electronic Notes in Discrete Mathematics* 202, 347–355.
- Zhang, D., Gu, C., Fang, H., 2022. Multi-strategy hybrid heuristic algorithm for single container weakly heterogeneous loading problem. *Computers and Industrial Engineering* 170, 108302.
- Zhang, D., Peng, Y., Leung, S.C.H., 2012. A heuristic block-loading algorithm based on multi-layer search for the container loading problem. *Computers and Operations Research* 39, 2267–2276.
- Zhao, X., Bennell, J.A., Bektaş, T., 2014. A comparative review of 3D container loading algorithms. *International Transactions in Operational Research* 23, 287–320.
- Zhu, W., Huang, W., Lim, A., 2012. A prototype column generation strategy for the multiple container loading problem. *European Journal of Operational Research* 223(1), 27–39.

Appendix A: Pseudocode of the algorithms proposed in this research

A. 1. Block generation algorithm

Algorithm 1 Block generation algorithm

Require: The length, width, and height of one given container k , the data frame of boxes **BoxList** (abbreviated as **BL**)

Ensure: The block list **BlockList** (abbreviated as **BLL**)

```

1:   Initialise BL = [];
2:   for  $i = 1$  to size(BL, 1) do
3:     for  $n_x = 1$  to BL( $i$ , 7) do
4:       for  $n_y = 1$  to floor(BL( $i$ , 7) /  $n_x$ ) do
5:         for  $n_z = 1$  to floor(BL( $i$ , 7) /  $n_x$  /  $n_y$ ) do
6:           if  $n_x * \mathbf{BL}(i, 1) \leq L_k$  and  $n_y * \mathbf{BL}(i, 2) \leq W_k$  and  $n_z * \mathbf{BL}(i, 3) \leq H_k$  then
7:             Add [ $n_x * \mathbf{BL}(i, 1)$ ,  $n_y * \mathbf{BL}(i, 2)$ ,  $n_z * \mathbf{BL}(i, 3)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
8:           end if
9:           if BL( $i$ , 1)1 · BL( $i$ , 2) then
10:            if  $n_x * \mathbf{BL}(i, 2) \leq L_k$  and  $n_y * \mathbf{BL}(i, 1) \leq W_k$  and  $n_z * \mathbf{BL}(i, 3) \leq H_k$  then
11:              Add [ $n_x * \mathbf{BL}(i, 2)$ ,  $n_y * \mathbf{BL}(i, 1)$ ,  $n_z * \mathbf{BL}(i, 3)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
12:            end if
13:          end if
14:          if BL( $i$ , 4) = 1 then
15:            if  $n_x * \mathbf{BL}(i, 2) \leq L_k$  and  $n_y * \mathbf{BL}(i, 3) \leq W_k$  and  $n_z * \mathbf{BL}(i, 1) \leq H_k$  then
16:              Add [ $n_x * \mathbf{BL}(i, 2)$ ,  $n_y * \mathbf{BL}(i, 3)$ ,  $n_z * \mathbf{BL}(i, 1)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
17:            end if
18:            if BL( $i$ , 2)1 · BL( $i$ , 3) then
19:              if  $n_x * \mathbf{BL}(i, 3) \leq L_k$  and  $n_y * \mathbf{BL}(i, 2) \leq W_k$  and  $n_z * \mathbf{BL}(i, 1) \leq H_k$  then
20:                Add [ $n_x * \mathbf{BL}(i, 3)$ ,  $n_y * \mathbf{BL}(i, 2)$ ,  $n_z * \mathbf{BL}(i, 1)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
21:              end if
22:            end if
23:          end if
24:          if BL( $i$ , 5) = 1 then
25:            if  $n_x * \mathbf{BL}(i, 1) \leq L_k$  and  $n_y * \mathbf{BL}(i, 3) \leq W_k$  and  $n_z * \mathbf{BL}(i, 2) \leq H_k$  then
26:              Add [ $n_x * \mathbf{BL}(i, 1)$ ,  $n_y * \mathbf{BL}(i, 3)$ ,  $n_z * \mathbf{BL}(i, 2)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
27:            end if
28:            if BL( $i$ , 1)1 · BL( $i$ , 3) then
29:              if  $n_x * \mathbf{BL}(i, 3) \leq L_k$  and  $n_y * \mathbf{BL}(i, 1) \leq W_k$  and  $n_z * \mathbf{BL}(i, 2) \leq H_k$  then
30:                Add [ $n_x * \mathbf{BL}(i, 3)$ ,  $n_y * \mathbf{BL}(i, 1)$ ,  $n_z * \mathbf{BL}(i, 2)$ ,  $n_x * n_y * n_z$ , BL( $i$ , 9), BL( $i$ , 10), BL( $i$ , 11)] to BLL;
31:              end if
32:            end if
33:          end if
34:        end for
35:      end for
36:    end for
37:  end for
38:  return BLL;

```

A. 2. Greedy heuristic multilayer tree search algorithm**Algorithm 2** Greedy heuristic multilayer tree search algorithm**Require:** $L_k, W_k, H_k, \text{BlockList}, \text{RootHeap}, \text{SubHeap}, \mathbb{C}, \text{mimolume}$ **Ensure:** *SearchTree* {1:3, 1} and the max loading rate

```

1:   Initialise ResidualSpace =  $\{L_k, W_k, H_k\}$ , LoadedBlocksList = [], SearchTree = {}, Child = {}, Index = [],
      curSpace = []
2:   for layer = 1 to  $\mathbb{C}$  do
3:     interBlockList = [];
4:     Add all block data belonging to layer priority in BlockList to interBlockList;
5:     if layer = 1 then
6:       [Index, curSpace] = [ResidualSpace, LoadedBlocksList, interBlockList,  $L_k, W_k, H_k, \text{mimolume}, \text{layer}, c$ ]
      out-put by Algorithm 5;
7:       for i = 1 to RootHeap do
8:         [newSpace, Referepointc] = [curSpace, LoadedBlocksList, Index(i)] output by Algorithm 3;
9:         SearchTree {1, i} = [SearchTree {1, i}; newSpace];
10:        SearchTree {2, i} = [interBlockList (Index(i), :), Referepointc]; // Referepointc is output by Algorithm 3;
11:        SearchTree {3, i} = SimplifyData(SearchTree {1, i}, interBlockList, Index(i))
12:      end for
13:    end if
14:    Index = 0;
15:    While Index  $\neq$  null do
16:      for i = 1 to RootHeap do
17:        [Index, curSpace] = [SearchTree {1, i}, SearchTree {2, i}, SearchTree {3, i},  $L_k, W_k, H_k, \text{mimolume}, \text{layer}, c$ ]
        output by Algorithm 5;
18:      for j = 1 to SubHeap do
19:        [newSpace, Referepointc] = [curSpace, SearchTree {3, i}, Index(j)] output by Algorithm 3;
20:        Delete curSpace from Child {1, (i-1)*SubHeap+j} and add newSpace to Child {1, (i-1)*SubHeap+j};
21:        Child {2, (i-1)*SubHeap+j} = [SearchTree {3, i} (Index(j), :), Referepointc];
22:        Child {3, (i-1)*SubHeap+j} = [Child {1, (i-1)*SubHeap+j}, Child {1, (i-1)*SubHeap+j}, Index(j)];
23:      end for
24:    end for
25:    Calculate the loading rate and sort Child in descending order of loading rate;
26:    SearchTree = Child {1: 3, 1: RootHeap};
27:  end while
28: end for
29: Return SearchTree {1:3, 1} and the max loading rate;

```

A. 3. Block loading algorithm**Algorithm 3** Block loading algorithm**Require:** *curSpace* (abbreviated as *cS*), *interBlockList*, *index***Ensure:** *newSpace*, *Referepointc*

```

1:  $l_x^d = cS(1) - \text{interBlockList}(\text{index}, 1);$ 
2:  $l_y^d = cS(2) - \text{interBlockList}(\text{index}, 2);$ 
3:  $l_z^d = cS(3) - \text{interBlockList}(\text{index}, 3);$ 
4: if  $l_y^d \leq l_x^d$  then
5:    $\text{newSpace}(1,1:7) = [l_x^d, cS(2), cS(3), cS(4) + \text{interBlockList}(\text{index}, 1), cS(5), cS(6), 1];$ 
6:    $\text{newSpace}(2,1:7) = [\text{interBlockList}(\text{index}, 1), l_y^d, cS(3), cS(4), cS(5) + \text{interBlockList}(\text{index}, 2), cS(6), 2];$ 
7: else
8:    $\text{newSpace}(1,1:7) = [l_x^d, \text{interBlockList}(\text{index}, 2), cS(3), cS(4) + \text{interBlockList}(\text{index}, 1), cS(5), cS(6), 1];$ 
9:    $\text{newSpace}(2,1:7) = [cS(1), l_y^d, cS(3), cS(4), cS(5) + \text{interBlockList}(\text{index}, 2), cS(6), 2];$ 
10: end if
11:  $\text{newSpace}(3,1:7) = [\text{interBlockList}(\text{index}, 1), \text{interBlockList}(\text{index}, 2), l_z^d, cS(4), cS(5), cS(6) +$ 
    $\text{interBlockList}(\text{index}, 3), 3];$ 
12:  $\text{Referepointc} = cS(4 : 6);$ 
13: Return newSpace, Referepointc;

```

A. 4. Block selection algorithm I**Algorithm 4** Block selection algorithm I**Require:** *interBlockList*, *curSpace* (abbreviated as *cS*), *minvolume***Ensure:** *interIndex*

```

1:   Initialise proIndex = [], interIndex = [];
2:   for i = 1 to size(interBlockList, 1) do
3:     if interBlockList(i,1) ≤ cS(1) and interBlockList(i,2) ≤ cS(2) and interBlockList(i,3) ≤ cS(3) then
4:       Add i to proIndex;
5:     end if
6:   end for
7:   if proIndex ≠ null then
8:     for j = 1 to length(proIndex) do
9:       [newSpace, Referepointc] = [cS, interBlocksList, proIndex(j)] output by Algorithm 3;
10:      tmp = 0;
11:      for k = 1 to 3 do
12:        if min(interBlocksList(:, k)) > newSpace(k, k) then
13:          if newSpace(k, 1) * newSpace(k, 2) * newSpace(k, 3) ≤ minvolume then
14:            tmp = tmp + 1;
15:          end if
16:        else
17:          tmp = tmp + 1;
18:        end if
19:      end for
20:      if tmp = 3 then
21:        Add proIndex(j) to interIndex;
22:      end if
23:    end for
24:  end if
25:  Return interIndex;

```

A. 5. Block selection algorithm II**Algorithm 5** Block selection algorithm II**Require:** *ResidualSpace*, *LoadedBlocksList*, *interBlockList*, *Container*, L_k , W_k , H_k , *minvolume*, *layer*, $\mathbb{C}$ **Ensure:** *interIndex*, *curSpace*

```

1:   Sort the data in ResidualSpace in descending order of  $re_x$ ,  $re_y$ ,  $re_z$  and  $s_{xyz}$ ;
2:   for  $i = 1$  to size(ResidualSpace, 1) do
3:     curSpace = ResidualSpace( $i, :$ );
4:     interIndex = GeninterIndexList(interBlockList, curSpace, minvolume);
5:     if interIndex  $\neq$  null then
6:       Region(1,1:6) = [curSpace(4:5), curSpace(6)+curSpace(3), curSpace(1:2)+curSpace(4:5), Container(3)];
7:       Region(2,1:6) = [curSpace(4)+curSpace(1), curSpace(5:6), Container(1), curSpace(2:3)+curSpace(5:6)];
8:       if LoadedBlocksList  $\neq$  null then
9:         tmp = 0;
10:        for  $j = 1$  to 2 do
11:          for  $k = 1$  to size(LoadedBlocksList, 1) do
12:            intermax = [max(Region( $j$ , 1), LoadedBlocksList( $k$ , 1)), max(Region( $j$ , 2), LoadedBlocksList( $k$ , 2)),
max(Region( $j$ , 3), LoadedBlocksList( $k$ , 3))];
13:            intermin = [min(Region( $j$ , 4), LoadedBlocksList( $k$ , 4)), min(Region( $j$ , 5), LoadedBlocksList( $k$ , 5)),
min(Region( $j$ , 6), LoadedBlocksList( $k$ , 6))];
14:            if intermax(1)  $\geq$  intermin(1) and intermax(2)  $\geq$  intermin(2) and intermax(3)  $\geq$  intermin(3) then
15:              tmp = tmp + 1;
16:            else if LoadedBlocksList( $k$ , 7)  $\leq$  (c - layer + 1) then
17:              tmp = tmp + 1;
18:            end if
19:          end for
20:        end for
21:        if tmp = 2 * size(LoadedBlocksList, 1) then
22:          break;
23:        else
24:          interIndex = [];
25:        end if
26:      end if
27:    end if
28:  end for
29:  Return interIndex, curSpace;

```

A. 6. Complex box identifying algorithm

Algorithm 6 Complex box identifying algorithm

Require: *BoxList*, *complexedge*, *complexvolume*
Ensure: *proBoxList*

- 1: Calculate the total volume *Totalvolume* of all boxes in *BoxList*;
 - 2: Count the orders with the cumulative volume of the boxes greater than $0.5 * \textit{Totalvolume}$ as the low loading priority, and add the corresponding index to *Lowlp*;
 - 3: *proBoxList* = [];
 - 4: **for** $i = 1$ **to** $\text{length}(\textit{Lowlp})$ **do**
 - 5: **if** $\max(\textit{BoxList}(\textit{Lowlp}(i), 1:3)) \geq \textit{complexedge}$ **then**
 - 6: Add *BoxList*(*Lowlp*(*i*),:) to *proBoxList*;
 - 7: **end if**
 - 8: **if** $\textit{BoxList}(\textit{Lowlp}(i), 1) * \textit{BoxList}(\textit{Lowlp}(i), 2) * \textit{BoxList}(\textit{Lowlp}(i), 3) \geq \textit{complexvolume}$ **then**
 - 9: Add *BoxList*(*Lowlp*(*i*),:) to *proBoxList*;
 - 10: **end if**
 - 11: **end for**
 - 12: **Return** *proBoxList*;
-

A. 7. Block loading algorithm with complex boxes

Algorithm 7 Block loading algorithm with complex boxes

Require: *curSpace* (abbreviated as *cS*), *interBlockList*, *index*
Ensure: *newSpace*, *Referepointc*

- 1: $l_x^d = cS(1) - \textit{interBlockList}(\textit{index}, 1)$;
 - 2: $l_y^d = cS(2) - \textit{interBlockList}(\textit{index}, 2)$;
 - 3: $l_z^d = cS(3) - \textit{interBlockList}(\textit{index}, 3)$;
 - 4: **if** $l_x^d \leq l_y^d$ **then**
 - 5: $\textit{newSpace}(1, 1:7) = [l_x^d, cS(2), cS(3), cS(4), cS(5), cS(6), 1]$;
 - 6: $\textit{newSpace}(2, 1:7) = [\textit{interBlockList}(\textit{index}, 1), l_y^d, cS(3), cS(4) + l_x^d, cS(5) + \textit{interBlockList}(\textit{index}, 2), cS(6), 2]$;
 - 7: **else**
 - 8: $\textit{newSpace}(1, 1:7) = [l_x^d, \textit{interBlockList}(\textit{index}, 2), cS(3), cS(4), cS(5), cS(6), 1]$;
 - 9: $\textit{newSpace}(2, 1:7) = [cS(1), l_y^d, cS(3), cS(4), cS(5) + \textit{interBlockList}(\textit{index}, 2), cS(6), 2]$;
 - 10: **end if**
 - 11: $\textit{newSpace}(3, 1:7) = [\textit{interBlockList}(\textit{index}, 1), \textit{interBlockList}(\textit{index}, 2), l_z^d, cS(4) + l_x^d, cS(5), cS(6) + \textit{interBlockList}(\textit{index}, 3), 3]$;
 - 12: $\textit{Referepointc} = [cS(4) + l_x^d, cS(5:6)]$;
 - 13: **Return** *newSpace*, *Referepointc*;
-

A. 8. Large neighborhood search**Algorithm 8** Large neighborhood search**Require:** initial solution x , maximum number of iterations NS **Ensure:** final solution

```

1:   $ns = 1$ ;
2:  while  $ns \leq NS$ 
3:    Obtain current solution  $x^b$ ;
4:    Choose a removal operator based on their likelihood of being selected;
5:    Apply the selected removal operator to destroy a part of  $x^b$ ;
6:    Choose a repair operator based on their likelihood of being selected;
7:    Apply the selected repair operator to repair the destroyed part of  $x^b$  to generate solution  $x^n$ ;
8:    if loading rate of  $x^n >$  loading rate of  $x^b$ 
9:       $x^b = x^n$ ;
10:   end if
11:    $ns = ns + 1$ ;
12: end while
13: final solution =  $x^b$ ;
14: Return final solution;

```
